



Built-in Functions

The Python 3 Standard Library

PROFESSIONAL REFERENCE MANUAL

Table of Contents

abs()
aiter()
all()
awaitable anext()
any()
ascii()
bin()
class bool()
breakpoint()
callable()
chr()
@classmethod()
compile()
class complex()
delattr()
dir()
divmod()
enumerate()
eval()
exec()
filter()
class float()
format()
getattr()
globals()
hasattr()
hash()
help()
hex()
id()
input()
class int()

isinstance()
issubclass()
iter()
len()
locals()
map()
max()
min()
next()
class object()
oct()
open()
ord()
pow()
print()
class property()
@getter()
@setter()
@deleter()
repr()
reversed()
round()
setattr()
class slice()
sorted()
@staticmethod()
sum()
class super()
class type()
vars()
zip()
__import__()

Built-in Functions

The Python interpreter has a number of functions and types built into it that are always available. They are listed here in alphabetical order.

Built-in Functions			
A abs() aiter() all() anext() any() ascii()	E enumerate() eval() exec()	L len() list() locals()	R range() repr() reversed() round()
B bin() bool() breakpoint() bytearray() bytes()	F filter() float() format() frozenset()	M map() max() memoryview() min()	S set() setattr() slice() sorted() staticmethod() str() sum() super()
C callable() chr() classmethod() compile() complex()	G getattr() globals()	N next()	T tuple() type()
D delattr() dict() dir() divmod()	H hasattr() hash() help() hex()	O object() oct() open() ord()	V vars()
	I id() input() int() isinstance() issubclass() iter()	P pow() print() property()	Z zip()
			- __import__()

abs(number, /)

Return the absolute value of a number. The argument may be an integer, a floating-point number, or an object implementing [__abs__\(\)](#) . If the argument is a complex number, its magnitude is returned.

`aiter(async_iterable, /)`

Return an [asynchronous iterator](#) for an [asynchronous iterable](#). Equivalent to calling `x.__aiter__()`.

Note: Unlike `iter()`, `aiter()` has no 2-argument variant.

Added in version 3.10.

`all(iterable, /)`

Return `True` if all elements of the *iterable* are true (or if the iterable is empty). Equivalent to:

```
def all(iterable):
    for element in iterable:
        if not element:
            return False
    return True
```

`awaitable anext(async_iterator, /)`

`awaitable anext(async_iterator, default, /)`

When awaited, return the next item from the given [asynchronous iterator](#), or *default* if given and the iterator is exhausted.

This is the async variant of the `next()` builtin, and behaves similarly.

This calls the `__anext__()` method of *async_iterator*, returning an [awaitable](#). Awaiting this returns the next value of the iterator. If *default* is given, it is returned if the iterator is exhausted, otherwise [StopAsyncIteration](#) is raised.

Added in version 3.10.

`any(iterable, /)`

Return `True` if any element of the *iterable* is true. If the iterable is empty, return `False`. Equivalent to:

```
def any(iterable):
    for element in iterable:
        if element:
            return True
    return False
```

`ascii(object, /)`

As `repr()`, return a string containing a printable representation of an object, but escape the non-ASCII characters in the string returned by `repr()` using `\x`, `\u`, or `\U` escapes. This generates a string similar to that returned by `repr()` in Python 2.

`bin(integer, /)`

Convert an integer number to a binary string prefixed with “0b”. The result is a valid Python expression. If *integer* is not a Python `int` object, it has to define an `__index__()` method that returns an integer. Some examples:

```
>>> bin(3)
'0b11'
>>> bin(-10)
'-0b1010'
```

If the prefix “0b” is desired or not, you can use either of the following ways.

```
>>> format(14, '#b'), format(14, 'b')
('0b1110', '1110')
>>> f'{14:#b}', f'{14:b}'
('0b1110', '1110')
```

See also `enum.bin()` to represent negative values as twos-complement.

See also `format()` for more information.

`class bool(object=False, /)`

Return a Boolean value, i.e. one of `True` or `False`. The argument is converted using the standard [truth testing procedure](#). If the argument is false or omitted, this returns `False`; otherwise, it returns `True`. The `bool` class is a subclass of `int` (see [Numeric Types — int, float, complex](#)). It cannot be subclassed further. Its only instances are `False` and `True` (see [Boolean Type - bool](#)).

Changed in version 3.7: The parameter is now positional-only.

`breakpoint(*args, **kws)`

This function drops you into the debugger at the call site. Specifically, it calls `sys.breakpointhook()`, passing `args` and `kws` straight through. By default, `sys.breakpointhook()` calls `pdb.set_trace()` expecting no arguments. In this case, it is purely a convenience function so you don't have to explicitly import `pdb` or type as much code to enter the debugger. However, `sys.breakpointhook()` can be set to some other function and `breakpoint()` will automatically call that, allowing you to drop into the debugger of choice. If `sys.breakpointhook()` is not accessible, this function will raise `RuntimeError`.

By default, the behavior of `breakpoint()` can be changed with the `PYTHONBREAKPOINT` environment variable. See `sys.breakpointhook()` for usage details.

Note that this is not guaranteed if `sys.breakpointhook()` has been replaced.

Raises an [auditing event](#) `builtins.breakpoint` with argument `breakpointhook`.

Added in version 3.7.

```
class bytearray(source=b'')
```

```
class bytearray(source, encoding, errors='strict')
```

Return a new array of bytes. The `bytearray` class is a mutable sequence of integers in the range $0 \leq x < 256$. It has most of the usual methods of mutable sequences, described in [Mutable Sequence Types](#), as well as most methods that the `bytes` type has, see [Bytes and Bytearray Operations](#).

The optional `source` parameter can be used to initialize the array in a few different ways:

- If it is a *string*, you must also give the *encoding* (and optionally, *errors*) parameters; `bytearray()` then converts the string to bytes using `str.encode()`.
- If it is an *integer*, the array will have that size and will be initialized with null bytes.
- If it is an object conforming to the [buffer interface](#), a read-only buffer of the object will be used to initialize the bytes array.
- If it is an *iterable*, it must be an iterable of integers in the range $0 \leq x < 256$, which are used as the initial contents of the array.

Without an argument, an array of size 0 is created.

See also [Binary Sequence Types — bytes, bytearray, memoryview](#) and [Bytearray Objects](#).

```
class bytes(source=b'')
```

```
class bytes(source, encoding, errors='strict')
```

Return a new “bytes” object which is an immutable sequence of integers in the range $0 \leq x < 256$. `bytes` is an immutable version of `bytearray` – it has the same non-mutating methods and the same indexing and slicing behavior.

Accordingly, constructor arguments are interpreted as for `bytearray()`.

Bytes objects can also be created with literals, see [String and Bytes literals](#).

See also [Binary Sequence Types — bytes, bytearray, memoryview](#), [Bytes Objects](#), and [Bytes and Bytearray Operations](#).

`callable(object, /)`

Return `True` if the *object* argument appears callable, `False` if not. If this returns `True`, it is still possible that a call fails, but if it is `False`, calling *object* will never succeed. Note that classes are callable (calling a class returns a new instance); instances are callable if their class has a `__call__()` method.

Added in version 3.2: This function was first removed in Python 3.0 and then brought back in Python 3.2.

`chr(codepoint, /)`

Return the string representing a character with the specified Unicode code point. For example, `chr(97)` returns the string `'a'`, while `chr(8364)` returns the string `'€'`. This is the inverse of `ord()`.

The valid range for the argument is from 0 through 1,114,111 (0x10FFFF in base 16). `ValueError` will be raised if it is outside that range.

@classmethod

Transform a method into a class method.

A class method receives the class as an implicit first argument, just like an instance method receives the instance. To declare a class method, use this idiom:

```
class C:
    @classmethod
    def f(cls, arg1, arg2): ...
```

The `@classmethod` form is a function [decorator](#) – see [Function definitions](#) for details.

A class method can be called either on the class (such as `C.f()`) or on an instance (such as `C().f()`). The instance is ignored except for its class. If a class method is called for a derived class, the derived class object is passed as the implied first argument.

Class methods are different than C++ or Java static methods. If you want those, see `staticmethod()` in this section. For more information on class methods, see [The standard type hierarchy](#).

Changed in version 3.9: Class methods can now wrap other [descriptors](#) such as `property()`.

Changed in version 3.10: Class methods now inherit the method attributes (`__module__`, `__name__`, `__qualname__`, `__doc__` and `__annotations__`) and have a new `__wrapped__` attribute.

Deprecated since version 3.11, removed in version 3.13: Class methods can no longer wrap other [descriptors](#) such as `property()`.

```
compile(source, filename, mode, flags=0, dont_inherit=False, optimize=-1)
```

Compile the *source* into a code or AST object. Code objects can be executed by `exec()` or `eval()`. *source* can either be a normal string, a byte string, or an AST object. Refer to the `ast` module documentation for information on how to work with AST objects.

The *filename* argument should give the file from which the code was read; pass some recognizable value if it wasn't read from a file (`'<string>'` is commonly used).

The *mode* argument specifies what kind of code must be compiled; it can be `'exec'` if *source* consists of a sequence of statements, `'eval'` if it consists of a single expression, or `'single'` if it consists of a single interactive statement (in the latter case, expression statements that evaluate to something other than `None` will be printed).

The optional arguments *flags* and *dont_inherit* control which [compiler options](#) should be activated and which [future features](#) should be allowed. If neither is present (or both are zero) the code is compiled with the same flags that affect the code that is calling `compile()`. If the *flags* argument is given and *dont_inherit* is not (or is zero) then the compiler options and the future statements specified by the *flags* argument are used in addition to those that would be used anyway. If *dont_inherit* is a non-zero integer then the *flags* argument is it – the flags (future features and compiler options) in the surrounding code are ignored.

Compiler options and future statements are specified by bits which can be bitwise ORed together to specify multiple options. The bitfield required to specify a given future feature can be found as the `compiler_flag` attribute on the `_Feature` instance in the `__future__` module. [Compiler flags](#) can be found in `ast` module, with `PyCF_` prefix.

The argument *optimize* specifies the optimization level of the compiler; the default value of `-1` selects the optimization level of the interpreter as given by `-O` options. Explicit levels are `0` (no optimization; `__debug__` is true), `1` (asserts are removed, `__debug__` is false) or `2` (docstrings are removed too).

This function raises `SyntaxError` or `ValueError` if the compiled source is invalid.

If you want to parse Python code into its AST representation, see `ast.parse()`.

Raises an `auditing_event compile` with arguments `source` and `filename`. This event may also be raised by implicit compilation.

Note: When compiling a string with multi-line code in `'single'` or `'eval'` mode, input must be terminated by at least one newline character. This is to facilitate detection of incomplete and complete statements in the `code` module.

Warning: It is possible to crash the Python interpreter with a sufficiently large/complex string when compiling to an AST object due to stack depth limitations in Python's AST compiler.

Changed in version 3.2: Allowed use of Windows and Mac newlines. Also, input in `'exec'` mode does not have to end in a newline anymore. Added the *optimize* parameter.

Changed in version 3.5: Previously, `TypeError` was raised when null bytes were encountered in *source*.

Added in version 3.8: `ast.PyCF_ALLOW_TOP_LEVEL_AWAIT` can now be passed in flags to enable support for top-level `await`, `async for`, and `async with`.

```
class complex(number=0, /)
```

```
class complex(string, /)
```

```
class complex(real=0, imag=0)
```

Convert a single string or number to a complex number, or create a complex number from real and imaginary parts.

Examples:

```
>>> complex('+1.23')
(1.23+0j)
>>> complex('-4.5j')
-4.5j
>>> complex('-1.23+4.5j')
(-1.23+4.5j)
>>> complex('\t( -1.23+4.5j )\n')
(-1.23+4.5j)
>>> complex('-Infinity+NaNj')
(-inf+nanj)
>>> complex(1.23)
(1.23+0j)
>>> complex(imag=-4.5)
-4.5j
>>> complex(-1.23, 4.5)
(-1.23+4.5j)
```

If the argument is a string, it must contain either a real part (in the same format as for `float()`) or an imaginary part (in the same format but with a `'j'` or `'J'` suffix), or both real and imaginary parts (the sign of the imaginary part is mandatory in this case). The string can optionally be surrounded by whitespaces and the round parentheses `'('` and `')'`, which are ignored. The string must not contain whitespace between `'+'`, `'-'`, the `'j'` or `'J'` suffix, and the decimal number. For example, `complex('1+2j')` is fine, but `complex('1 + 2j')` raises `ValueError`. More precisely, the input must conform to the `complexvalue` production rule in the following grammar, after parentheses and leading and trailing whitespace characters are removed:

```
complexvalue: floatvalue |
              floatvalue ("j" | "J") |
              floatvalue sign absfloatvalue ("j" | "J")
```

If the argument is a number, the constructor serves as a numeric conversion like `int` and `float`. For a general Python object `x`, `complex(x)` delegates to `x.__complex__()`. If `__complex__()` is not defined then it falls back to `__float__()`. If `__float__()` is not defined then it falls back to `__index__()`.

If two arguments are provided or keyword arguments are used, each argument may be any numeric type (including complex). If both arguments are real numbers, return a complex number with the real component `real` and the imaginary component `imag`. If both arguments are complex numbers, return a complex number with the real component `real.real-imag.imag` and the imaginary component `real.imag+imag.real`. If one of arguments is a real number, only its real component is used in the above expressions.

See also `complex.from_number()` which only accepts a single numeric argument.

If all arguments are omitted, returns `0j`.

The complex type is described in [Numeric Types — int, float, complex](#).

Changed in version 3.6: Grouping digits with underscores as in code literals is allowed.

Changed in version 3.8: Falls back to `__index__()` if `__complex__()` and `__float__()` are not defined.

Deprecated since version 3.14: Passing a complex number as the *real* or *imag* argument is now deprecated; it should only be passed as a single positional argument.

`delattr(object, name, /)`

This is a relative of `setattr()`. The arguments are an object and a string. The string must be the name of one of the object's attributes. The function deletes the named attribute, provided the object allows it. For example, `delattr(x, 'foobar')` is equivalent to `del x.foobar`. *name* need not be a Python identifier (see `setattr()`).

`class dict(**kwargs)`

`class dict(mapping, /, **kwargs)`

`class dict(iterable, /, **kwargs)`

Create a new dictionary. The `dict` object is the dictionary class. See `dict` and [Mapping Types — dict](#) for documentation about this class.

For other containers see the built-in `list`, `set`, and `tuple` classes, as well as the `collections` module.

dir()

dir(object, Δ)

Without arguments, return the list of names in the current local scope. With an argument, attempt to return a list of valid attributes for that object.

If the object has a method named `__dir__()`, this method will be called and must return the list of attributes. This allows objects that implement a custom `__getattr__()` or `__getattribute__()` function to customize the way `dir()` reports their attributes.

If the object does not provide `__dir__()`, the function tries its best to gather information from the object's `__dict__` attribute, if defined, and from its type object. The resulting list is not necessarily complete and may be inaccurate when the object has a custom `__getattr__()`.

The default `dir()` mechanism behaves differently with different types of objects, as it attempts to produce the most relevant, rather than complete, information:

- If the object is a module object, the list contains the names of the module's attributes.
- If the object is a type or class object, the list contains the names of its attributes, and recursively of the attributes of its bases.
- Otherwise, the list contains the object's attributes' names, the names of its class's attributes, and recursively of the attributes of its class's base classes.

The resulting list is sorted alphabetically. For example:

```
>>> import struct
>>> dir()    # show the names in the module namespace
['__builtin__', '__name__', 'struct']
>>> dir(struct)    # show the names in the struct module
['Struct', '__all__', '__builtin__', '__cached__', '__doc__', '__file__',
 '__initializing__', '__loader__', '__name__', '__package__',
 '__clearcache__', 'calcsizes', 'error', 'pack', 'pack_into',
 'unpack', 'unpack_from']
>>> class Shape:
...     def __dir__(self):
...         return ['area', 'perimeter', 'location']
...
>>> s = Shape()
>>> dir(s)
['area', 'location', 'perimeter']
```

Note: Because `dir()` is supplied primarily as a convenience for use at an interactive prompt, it tries to supply an interesting set of names more than it tries to supply a rigorously or consistently defined set of names, and its detailed behavior may change across releases. For example, metaclass attributes are not in the result list when the argument is a class.

`divmod(a, b, /)`

Take two (non-complex) numbers as arguments and return a pair of numbers consisting of their quotient and remainder when using integer division. With mixed operand types, the rules for binary arithmetic operators apply. For integers, the result is the same as `(a // b, a % b)`. For floating-point numbers the result is `(q, a % b)`, where q is usually `math.floor(a / b)` but may be 1 less than that. In any case `q * b + a % b` is very close to a , if `a % b` is non-zero it has the same sign as b , and `0 <= abs(a % b) < abs(b)`.

`enumerate(iterable, start=0)`

Return an enumerate object. *iterable* must be a sequence, an [iterator](#), or some other object which supports iteration. The `__next__()` method of the iterator returned by `enumerate()` returns a tuple containing a count (from *start* which defaults to 0) and the values obtained from iterating over *iterable*.

```
>>> seasons = ['Spring', 'Summer', 'Fall', 'Winter']
>>> list(enumerate(seasons))
[(0, 'Spring'), (1, 'Summer'), (2, 'Fall'), (3, 'Winter')]
>>> list(enumerate(seasons, start=1))
[(1, 'Spring'), (2, 'Summer'), (3, 'Fall'), (4, 'Winter')]
```

Equivalent to:

```
def enumerate(iterable, start=0):
    n = start
    for elem in iterable:
        yield n, elem
        n += 1
```


`eval(source, /, globals=None, locals=None)`

Parameters:

- **source** (`str` | `code object`) – A Python expression.
- **globals** (`dict` | `None`) – The global namespace (default: `None`).
- **locals** (`mapping` | `None`) – The local namespace (default: `None`).

Returns:

The result of the evaluated expression.

Raises:

Syntax errors are reported as exceptions.

Warning: This function executes arbitrary code. Calling it with user-supplied input may lead to security vulnerabilities.

The *source* argument is parsed and evaluated as a Python expression (technically speaking, a condition list) using the *globals* and *locals* mappings as global and local namespace. If the *globals* dictionary is present and does not contain a value for the key `__builtins__`, a reference to the dictionary of the built-in module `builtins` is inserted under that key before *source* is parsed. That way you can control what builtins are available to the executed code by inserting your own `__builtins__` dictionary into *globals* before passing it to `eval()`. If the *locals* mapping is omitted it defaults to the *globals* dictionary. If both mappings are omitted, the source is executed with the *globals* and *locals* in the environment where `eval()` is called. Note, `eval()` will only have access to the [nested scopes](#) (non-locals) in the enclosing environment if they are already referenced in the scope that is calling `eval()` (e.g. via a `nonlocal` statement).

Example:

```
>>> x = 1
>>> eval('x+1')
2
```

This function can also be used to execute arbitrary code objects (such as those created by `compile()`). In this case, pass a code object instead of a string. If the code object has been compiled with `'exec'` as the *mode* argument, `eval()`'s return value will be `None`.

Hints: dynamic execution of statements is supported by the `exec()` function. The `globals()` and `locals()` functions return the current global and local dictionary, respectively, which may be useful to pass around for use by `eval()` or `exec()`.

If the given source is a string, then leading and trailing spaces and tabs are stripped.

See `ast.literal_eval()` for a function that can safely evaluate strings with expressions containing only literals.

Raises an `auditing_event` `exec` with the code object as the argument. Code compilation events may also be raised.

Changed in version 3.13: The *globals* and *locals* arguments can now be passed as keywords.

Changed in version 3.13: The semantics of the default *locals* namespace have been adjusted as described for the `locals()` builtin.

`exec(source, /, globals=None, locals=None, *, closure=None)`

Warning: This function executes arbitrary code. Calling it with user-supplied input may lead to security vulnerabilities.

This function supports dynamic execution of Python code. *source* must be either a string or a code object. If it is a string, the string is parsed as a suite of Python statements which is then executed (unless a syntax error occurs). [1] If it is a code object, it is simply executed. In all cases, the code that's executed is expected to be valid as file input (see the section [File input](#) in the Reference Manual). Be aware that the `nonlocal`, `yield`, and `return` statements may not be used outside of function definitions even within the context of code passed to the `exec()` function. The return value is `None`.

In all cases, if the optional parts are omitted, the code is executed in the current scope. If only *globals* is provided, it must be a dictionary (and not a subclass of dictionary), which will be used for both the global and the local variables. If *globals* and *locals* are given, they are used for the global and local variables, respectively. If provided, *locals* can be any mapping object. Remember that at the module level, *globals* and *locals* are the same dictionary.

Note: When `exec` gets two separate objects as *globals* and *locals*, the code will be executed as if it were embedded in a class definition. This means functions and classes defined in the executed code will not be able to access variables assigned at the top level (as the “top level” variables are treated as class variables in a class definition).

If the *globals* dictionary does not contain a value for the key `__builtins__`, a reference to the dictionary of the built-in module `builtins` is inserted under that key. That way you can control what builtins are available to the executed code by inserting your own `__builtins__` dictionary into *globals* before passing it to `exec()`.

The *closure* argument specifies a closure—a tuple of cellvars. It's only valid when the *object* is a code object containing [free \(closure\) variables](#). The length of the tuple must exactly match the length of the code object's `co_freevars` attribute.

Raises an [auditing event](#) `exec` with the code object as the argument. Code compilation events may also be raised.

Note: The built-in functions `globals()` and `locals()` return the current global and local namespace, respectively, which may be useful to pass around for use as the second and third argument to `exec()`.

Note: The default *locals* act as described for function `locals()` below. Pass an explicit *locals* dictionary if you need to see effects of the code on *locals* after function `exec()` returns.

Changed in version 3.11: Added the *closure* parameter.

Changed in version 3.13: The *globals* and *locals* arguments can now be passed as keywords.

Changed in version 3.13: The semantics of the default *locals* namespace have been adjusted as described for the `locals()` builtin.

`filter(function, iterable, /)`

Construct an iterator from those elements of *iterable* for which *function* is true. *iterable* may be either a sequence, a container which supports iteration, or an iterator. If *function* is `None`, the identity function is assumed, that is, all elements of *iterable* that are false are removed.

Note that `filter(function, iterable)` is equivalent to the generator expression `(item for item in iterable if function(item))` if function is not `None` and `(item for item in iterable if item)` if function is `None`.

See [`itertools.filterfalse\(\)`](#) for the complementary function that returns elements of *iterable* for which *function* is false.

`class float(number=0.0, /)`

`class float(string, /)`

Return a floating-point number constructed from a number or a string.

Examples:

```
>>> float('+1.23')
1.23
>>> float('  -12345\n')
-12345.0
>>> float('1e-003')
0.001
>>> float('+1E6')
1000000.0
>>> float('-Infinity')
-inf
```

If the argument is a string, it should contain a decimal number, optionally preceded by a sign, and optionally embedded in whitespace. The optional sign may be `'+'` or `'-'`; a `'+'` sign has no effect on the value produced. The argument may also be a string representing a NaN (not-a-number), or positive or negative infinity. More precisely, the input must conform to the [floatvalue](#) production rule in the following grammar, after leading and trailing whitespace characters are removed:

```
sign:          "+" | "-"
infinity:      "Infinity" | "inf"
nan:           "nan"
digit:         <a Unicode decimal digit, i.e. characters in Unicode general category Nd>
digitpart:     digit ["_"] digit)*
number:        [digitpart] "." digitpart | digitpart ["," ]
exponent:      ("e" | "E") [sign] digitpart
floatnumber:   number [exponent]
absfloatvalue: floatnumber | infinity | nan
floatvalue:    [sign] absfloatvalue
```

Case is not significant, so, for example, `"inf"`, `"Inf"`, `"INFINITY"`, and `"iNfINity"` are all acceptable spellings for positive infinity.

Otherwise, if the argument is an integer or a floating-point number, a floating-point number with the same value (within Python's floating-point precision) is returned. If the argument is outside the range of a Python float, an [OverflowError](#) will be raised.

For a general Python object `x`, `float(x)` delegates to `x.__float__()`. If `__float__()` is not defined then it falls back to `__index__()`.

See also [float.from_number\(\)](#) which only accepts a numeric argument.

If no argument is given, `0.0` is returned.

The float type is described in [Numeric Types — int, float, complex](#).

Changed in version 3.6: Grouping digits with underscores as in code literals is allowed.

Changed in version 3.7: The parameter is now positional-only.

Changed in version 3.8: Falls back to `__index__()` if `__float__()` is not defined.

`format(value, format_spec='', /)`

Convert a *value* to a “formatted” representation, as controlled by *format_spec*. The interpretation of *format_spec* will depend on the type of the *value* argument; however, there is a standard formatting syntax that is used by most built-in types: [Format Specification Mini-Language](#).

The default *format_spec* is an empty string which usually gives the same effect as calling `str(value)`.

A call to `format(value, format_spec)` is translated to `type(value).__format__(value, format_spec)` which bypasses the instance dictionary when searching for the value’s `__format__()` method. A `TypeError` exception is raised if the method search reaches `object` and the *format_spec* is non-empty, or if either the *format_spec* or the return value are not strings.

Changed in version 3.4: `object().__format__(format_spec)` raises `TypeError` if *format_spec* is not an empty string.

`class frozenset(iterable=(), /)`

Return a new `frozenset` object, optionally with elements taken from *iterable*. `frozenset` is a built-in class. See `frozenset` and [Set Types — set, frozenset](#) for documentation about this class.

For other containers see the built-in `set`, `list`, `tuple`, and `dict` classes, as well as the [collections](#) module.

`getattr(object, name, /)`

`getattr(object, name, default, /)`

Return the value of the named attribute of *object*. *name* must be a string. If the string is the name of one of the object's attributes, the result is the value of that attribute. For example, `getattr(x, 'foobar')` is equivalent to `x.foobar`. If the named attribute does not exist, *default* is returned if provided, otherwise `AttributeError` is raised. *name* need not be a Python identifier (see `setattr()`).

Note: Since [private name mangling](#) happens at compilation time, one must manually mangle a private attribute's (attributes with two leading underscores) name in order to retrieve it with `getattr()`.

`globals()`

Return the dictionary implementing the current module namespace. For code within functions, this is set when the function is defined and remains the same regardless of where the function is called.

`hasattr(object, name, /)`

The arguments are an object and a string. The result is `True` if the string is the name of one of the object's attributes, `False` if not. (This is implemented by calling `getattr(object, name)` and seeing whether it raises an `AttributeError` or not.)

`hash(object, /)`

Return the hash value of the object (if it has one). Hash values are integers. They are used to quickly compare dictionary keys during a dictionary lookup. Numeric values that compare equal have the same hash value (even if they are of different types, as is the case for 1 and 1.0).

Note: For objects with custom `__hash__()` methods, note that `hash()` truncates the return value based on the bit width of the host machine.

help()

help(request)

Invoke the built-in help system. (This function is intended for interactive use.) If no argument is given, the interactive help system starts on the interpreter console. If the argument is a string, then the string is looked up as the name of a module, function, class, method, keyword, or documentation topic, and a help page is printed on the console. If the argument is any other kind of object, a help page on the object is generated.

Note that if a slash(/) appears in the parameter list of a function when invoking `help()`, it means that the parameters prior to the slash are positional-only. For more info, see [the FAQ entry on positional-only parameters](#).

This function is added to the built-in namespace by the `site` module.

Changed in version 3.4: Changes to `pydoc` and `inspect` mean that the reported signatures for callables are now more comprehensive and consistent.

hex(integer, /...)

Convert an integer number to a lowercase hexadecimal string prefixed with “0x”. If *integer* is not a Python `int` object, it has to define an `__index__()` method that returns an integer. Some examples:

```
>>> hex(255)
'0xff'
>>> hex(-42)
'-0x2a'
```

If you want to convert an integer number to an uppercase or lower hexadecimal string with prefix or not, you can use either of the following ways:

```
>>> '%#x' % 255, '%x' % 255, '%X' % 255
('0xff', 'ff', 'FF')
>>> format(255, '#x'), format(255, 'x'), format(255, 'X')
('0xff', 'ff', 'FF')
>>> f'{255:#x}', f'{255:x}', f'{255:X}'
('0xff', 'ff', 'FF')
```

See also `format()` for more information.

See also `int()` for converting a hexadecimal string to an integer using a base of 16.

Note: To obtain a hexadecimal string representation for a float, use the `float.hex()` method.

`id(object, /)`

Return the “identity” of an object. This is an integer which is guaranteed to be unique and constant for this object during its lifetime. Two objects with non-overlapping lifetimes may have the same `id()` value.

CPython implementation detail: This is the address of the object in memory.

Raises an [auditing event](#) `builtins.id` with argument `id`.

`input()`

`input(prompt, /)`

If the *prompt* argument is present, it is written to standard output without a trailing newline. The function then reads a line from input, converts it to a string (stripping a trailing newline), and returns that. When EOF is read, [EOFError](#) is raised. Example:

```
>>> s = input('--> ')
--> Monty Python's Flying Circus
>>> s
"Monty Python's Flying Circus"
```

If the [readline](#) module was loaded, then `input()` will use it to provide elaborate line editing and history features.

Raises an [auditing event](#) `builtins.input` with argument `prompt` before reading input

Raises an [auditing event](#) `builtins.input/result` with the result after successfully reading input.

```
class int(number=0, /)
```

```
class int(string, /, base=10)
```

Return an integer object constructed from a number or a string, or return `0` if no arguments are given.

Examples:

```
>>> int(123.45)
123
>>> int('123')
123
>>> int(' -12_345\n')
-12345
>>> int('FACE', 16)
64206
>>> int('0xface', 0)
64206
>>> int('01110011', base=2)
115
```

If the argument defines `__int__()`, `int(x)` returns `x.__int__()`. If the argument defines `__index__()`, it returns `x.__index__()`. For floating-point numbers, this truncates towards zero.

If the argument is not a number or if `base` is given, then it must be a string, `bytes`, or `bytearray` instance representing an integer in radix `base`. Optionally, the string can be preceded by `+` or `-` (with no space in between), have leading zeros, be surrounded by whitespace, and have single underscores interspersed between digits.

A base-`n` integer string contains digits, each representing a value from 0 to `n-1`. The values 0–9 can be represented by any Unicode decimal digit. The values 10–35 can be represented by `a` to `z` (or `A` to `Z`). The default `base` is 10. The allowed bases are 0 and 2–36. Base-2, -8, and -16 strings can be optionally prefixed with `0b` / `0B`, `0o` / `0O`, or `0x` / `0X`, as with integer literals in code. For base 0, the string is interpreted in a similar way to an [integer literal in code](#), in that the actual base is 2, 8, 10, or 16 as determined by the prefix. Base 0 also disallows leading zeros: `int('010', 0)` is not legal, while `int('010')` and `int('010', 8)` are.

The integer type is described in [Numeric Types — int, float, complex](#).

Changed in version 3.4: If `base` is not an instance of `int` and the `base` object has a `base.__index__` method, that method is called to obtain an integer for the base. Previous versions used `base.__int__` instead of `base.__index__`.

Changed in version 3.6: Grouping digits with underscores as in code literals is allowed.

Changed in version 3.7: The first parameter is now positional-only.

Changed in version 3.8: Falls back to `__index__()` if `__int__()` is not defined.

Changed in version 3.11: `int` string inputs and string representations can be limited to help avoid denial of service attacks. A `ValueError` is raised when the limit is exceeded while converting a string to an `int` or when converting an `int` into a string would exceed the limit. See the [integer string conversion length limitation](#) documentation.

Changed in version 3.14: `int()` no longer delegates to the `__trunc__()` method.

`isinstance(object, classinfo, /)`

Return `True` if the *object* argument is an instance of the *classinfo* argument, or of a (direct, indirect, or [virtual](#)) subclass thereof. If *object* is not an object of the given type, the function always returns `False`. If *classinfo* is a tuple of type objects (or recursively, other such tuples) or a [Union Type](#) of multiple types, return `True` if *object* is an instance of any of the types. If *classinfo* is not a type or tuple of types and such tuples, a [TypeError](#) exception is raised. [TypeError](#) may not be raised for an invalid type if an earlier check succeeds.

Changed in version 3.10: *classinfo* can be a [Union Type](#).

`issubclass(class, classinfo, /)`

Return `True` if *class* is a subclass (direct, indirect, or [virtual](#)) of *classinfo*. A class is considered a subclass of itself. *classinfo* may be a tuple of class objects (or recursively, other such tuples) or a [Union Type](#), in which case return `True` if *class* is a subclass of any entry in *classinfo*. In any other case, a [TypeError](#) exception is raised.

Changed in version 3.10: *classinfo* can be a [Union Type](#).

`iter(iterable, /)`

`iter(callable, sentinel, /)`

Return an [iterator](#) object. The first argument is interpreted very differently depending on the presence of the second argument. Without a second argument, the single argument must be a collection object which supports the [iterable](#) protocol (the `__iter__()` method), or it must support the sequence protocol (the `__getitem__()` method with integer arguments starting at `0`). If it does not support either of those protocols, `TypeError` is raised. If the second argument, *sentinel*, is given, then the first argument must be a callable object. The iterator created in this case will call *callable* with no arguments for each call to its `__next__()` method; if the value returned is equal to *sentinel*, `StopIteration` will be raised, otherwise the value will be returned.

See also [Iterator Types](#).

One useful application of the second form of `iter()` is to build a block-reader. For example, reading fixed-width blocks from a binary database file until the end of file is reached:

```
from functools import partial
with open('mydata.db', 'rb') as f:
    for block in iter(partial(f.read, 64), b''):
        process_block(block)
```

`len(object, /)`

Return the length (the number of items) of an object. The argument may be a sequence (such as a string, bytes, tuple, list, or range) or a collection (such as a dictionary, set, or frozen set).

CPython implementation detail: `len` raises `OverflowError` on lengths larger than `sys.maxsize`, such as `range(2 ** 100)`.

`class list(iterable=(), /)`

Rather than being a function, `list` is actually a mutable sequence type, as documented in [Lists](#) and [Sequence Types — list, tuple, range](#).

locals()

Return a mapping object representing the current local symbol table, with variable names as the keys, and their currently bound references as the values.

At module scope, as well as when using `exec()` or `eval()` with a single namespace, this function returns the same namespace as `globals()`.

At class scope, it returns the namespace that will be passed to the metaclass constructor.

When using `exec()` or `eval()` with separate local and global arguments, it returns the local namespace passed in to the function call.

In all of the above cases, each call to `locals()` in a given frame of execution will return the *same* mapping object. Changes made through the mapping object returned from `locals()` will be visible as assigned, reassigned, or deleted local variables, and assigning, reassigned, or deleting local variables will immediately affect the contents of the returned mapping object.

In an [optimized scope](#) (including functions, generators, and coroutines), each call to `locals()` instead returns a fresh dictionary containing the current bindings of the function's local variables and any nonlocal cell references. In this case, name binding changes made via the returned dict are *not* written back to the corresponding local variables or nonlocal cell references, and assigning, reassigned, or deleting local variables and nonlocal cell references does *not* affect the contents of previously returned dictionaries.

Calling `locals()` as part of a comprehension in a function, generator, or coroutine is equivalent to calling it in the containing scope, except that the comprehension's initialised iteration variables will be included. In other scopes, it behaves as if the comprehension were running as a nested function.

Calling `locals()` as part of a generator expression is equivalent to calling it in a nested generator function.

Changed in version 3.12: The behaviour of `locals()` in a comprehension has been updated as described in [PEP 709](#).

Changed in version 3.13: As part of [PEP 667](#), the semantics of mutating the mapping objects returned from this function are now defined. The behavior in [optimized scopes](#) is now as described above. Aside from being defined, the behaviour in other scopes remains unchanged from previous versions.

map(function, iterable, /, *iterables, strict=False)

Return an iterator that applies *function* to every item of *iterable*, yielding the results. If additional *iterables* arguments are passed, *function* must take that many arguments and is applied to the items from all iterables in parallel. With multiple iterables, the iterator stops when the shortest iterable is exhausted. If *strict* is `True` and one of the iterables is exhausted before the others, a `ValueError` is raised. For cases where the function inputs are already arranged into argument tuples, see [itertools.starmap\(\)](#).

Changed in version 3.14: Added the *strict* parameter.

`max(iterable, /, *, key=None)`

`max(iterable, /, *, default, key=None)`

`max(arg1, arg2, /, *args, key=None)`

Return the largest item in an iterable or the largest of two or more arguments.

If one positional argument is provided, it should be an [iterable](#). The largest item in the iterable is returned. If two or more positional arguments are provided, the largest of the positional arguments is returned.

There are two optional keyword-only arguments. The `key` argument specifies a one-argument ordering function like that used for `list.sort()`. The `default` argument specifies an object to return if the provided iterable is empty. If the iterable is empty and `default` is not provided, a [ValueError](#) is raised.

If multiple items are maximal, the function returns the first one encountered. This is consistent with other sort-stability preserving tools such as `sorted(iterable, key=keyfunc, reverse=True)[0]` and `heapq.nlargest(1, iterable, key=keyfunc)`.

Changed in version 3.4: Added the `default` keyword-only parameter.

Changed in version 3.8: The `key` can be `None`.

`class memoryview(object)`

Return a “memory view” object created from the given argument. See [Memory Views](#) for more information.

`min(iterable, /, *, key=None)`

`min(iterable, /, *, default, key=None)`

`min(arg1, arg2, /, *args, key=None)`

Return the smallest item in an iterable or the smallest of two or more arguments.

If one positional argument is provided, it should be an [iterable](#). The smallest item in the iterable is returned. If two or more positional arguments are provided, the smallest of the positional arguments is returned.

There are two optional keyword-only arguments. The `key` argument specifies a one-argument ordering function like that used for `list.sort()`. The `default` argument specifies an object to return if the provided iterable is empty. If the iterable is empty and `default` is not provided, a [ValueError](#) is raised.

If multiple items are minimal, the function returns the first one encountered. This is consistent with other sort-stability preserving tools such as `sorted(iterable, key=keyfunc)[0]` and `heapq.nsmallest(1, iterable, key=keyfunc)`.

Changed in version 3.4: Added the `default` keyword-only parameter.

Changed in version 3.8: The `key` can be `None`.

`next(iterator, /)`

`next(iterator, default, /)`

Retrieve the next item from the [iterator](#) by calling its `__next__()` method. If `default` is given, it is returned if the iterator is exhausted, otherwise [StopIteration](#) is raised.

`class object`

This is the ultimate base class of all other classes. It has methods that are common to all instances of Python classes. When the constructor is called, it returns a new featureless object. The constructor does not accept any arguments.

Note: `object` instances do *not* have `__dict__` attributes, so you can't assign arbitrary attributes to an instance of `object`.

`oct(integer, /)`

Convert an integer number to an octal string prefixed with “0o”. The result is a valid Python expression. If *integer* is not a Python `int` object, it has to define an `__index__()` method that returns an integer. For example:

```
>>> oct(8)
'0o10'
>>> oct(-56)
'-0o70'
```

If you want to convert an integer number to an octal string either with the prefix “0o” or not, you can use either of the following ways.

```
>>> '%#o' % 10, '%o' % 10
('0o12', '12')
>>> format(10, '#o'), format(10, 'o')
('0o12', '12')
>>> f'{10:#o}', f'{10:o}'
('0o12', '12')
```

See also `format()` for more information.


```
open(file, mode='r', buffering=-1, encoding=None, errors=None,
newline=None, closefd=True, opener=None)
```

Open *file* and return a corresponding [file object](#). If the file cannot be opened, an `OSError` is raised. See [Reading and Writing Files](#) for more examples of how to use this function.

file is a [path-like object](#) giving the pathname (absolute or relative to the current working directory) of the file to be opened or an integer file descriptor of the file to be wrapped. (If a file descriptor is given, it is closed when the returned I/O object is closed unless *closefd* is set to `False`.)

mode is an optional string that specifies the mode in which the file is opened. It defaults to `'r'` which means open for reading in text mode. Other common values are `'w'` for writing (truncating the file if it already exists), `'x'` for exclusive creation, and `'a'` for appending (which on *some* Unix systems, means that *all* writes append to the end of the file regardless of the current seek position). In text mode, if *encoding* is not specified the encoding used is platform-dependent: `locale.getencoding()` is called to get the current locale encoding. (For reading and writing raw bytes use binary mode and leave *encoding* unspecified.) The available modes are:

Character	Meaning
<code>'r'</code>	open for reading (default)
<code>'w'</code>	open for writing, truncating the file first
<code>'x'</code>	open for exclusive creation, failing if the file already exists
<code>'a'</code>	open for writing, appending to the end of file if it exists
<code>'b'</code>	binary mode
<code>'t'</code>	text mode (default)
<code>'+'</code>	open for updating (reading and writing)

The default mode is `'r'` (open for reading text, a synonym of `'rt'`). Modes `'w+'` and `'w+b'` open and truncate the file. Modes `'r+'` and `'r+b'` open the file with no truncation.

As mentioned in the [Overview](#), Python distinguishes between binary and text I/O. Files opened in binary mode (including `'b'` in the *mode* argument) return contents as [bytes](#) objects without any decoding. In text mode (the default, or when `'t'` is included in the *mode* argument), the contents of the file are returned as [str](#), the bytes having been first decoded using a platform-dependent encoding or using the specified *encoding* if given.

Note: Python doesn't depend on the underlying operating system's notion of text files; all the processing is done by Python itself, and is therefore platform-independent.

buffering is an optional integer used to set the buffering policy. Pass 0 to switch buffering off (only allowed in binary mode), 1 to select line buffering (only usable when writing in text mode), and an integer > 1 to indicate the size in bytes of a fixed-size chunk buffer. Note that specifying a buffer size this way applies for binary buffered I/O, but `TextIOWrapper` (i.e., files opened with `mode='r+'`) would have another buffering. To disable buffering in `TextIOWrapper`, consider using the `write_through` flag for [io.TextIOWrapper.reconfigure\(\)](#). When no *buffering* argument is given, the default buffering policy works as follows:

- Binary files are buffered in fixed-size chunks; the size of the buffer is `max(min(blocksize, 8 MiB), DEFAULT_BUFFER_SIZE)` when the device block size is available. On most systems, the buffer will typically be 128 kilobytes long.
- “Interactive” text files (files for which `isatty()` returns `True`) use line buffering. Other text files use the policy described above for binary files.

encoding is the name of the encoding used to decode or encode the file. This should only be used in text mode. The default encoding is platform dependent (whatever `locale.getencoding()` returns), but any [text encoding](#) supported by Python can be used. See the [codecs](#) module for the list of supported encodings.

errors is an optional string that specifies how encoding and decoding errors are to be handled—this cannot be used in binary mode. A variety of standard error handlers are available (listed under [Error Handlers](#)), though any error handling name that has been registered with `codecs.register_error()` is also valid. The standard names include:

- `'strict'` to raise a `ValueError` exception if there is an encoding error. The default value of `None` has the same effect.
- `'ignore'` ignores errors. Note that ignoring encoding errors can lead to data loss.
- `'replace'` causes a replacement marker (such as `'?'`) to be inserted where there is malformed data.
- `'surrogateescape'` will represent any incorrect bytes as low surrogate code units ranging from U+DC80 to U+DCFF. These surrogate code units will then be turned back into the same bytes when the `surrogateescape` error handler is used when writing data. This is useful for processing files in an unknown encoding.
- `'xmlcharrefreplace'` is only supported when writing to a file. Characters not supported by the encoding are replaced with the appropriate XML character reference `&#nnn;`.
- `'backslashreplace'` replaces malformed data by Python’s backslashed escape sequences.
- `'namereplace'` (also only supported when writing) replaces unsupported characters with `\N{...}` escape sequences.

newline determines how to parse newline characters from the stream. It can be `None`, `''`, `'\n'`, `'\r'`, and `'\r\n'`. It works as follows:

- When reading input from the stream, if *newline* is `None`, universal newlines mode is enabled. Lines in the input can end in `'\n'`, `'\r'`, or `'\r\n'`, and these are translated into `'\n'` before being returned to the caller. If it is `''`, universal newlines mode is enabled, but line endings are returned to the caller untranslated. If it has any of the other legal values, input lines are only terminated by the given string, and the line ending is returned to the caller untranslated.
- When writing output to the stream, if *newline* is `None`, any `'\n'` characters written are translated to the system default line separator, `os.linesep`. If *newline* is `''` or `'\n'`, no translation takes place. If *newline* is any of the other legal values, any `'\n'` characters written are translated to the given string.

If *closefd* is `False` and a file descriptor rather than a filename was given, the underlying file descriptor will be kept open when the file is closed. If a filename is given *closefd* must be `True` (the default); otherwise, an error will be raised.

A custom opener can be used by passing a callable as *opener*. The underlying file descriptor for the file object is then obtained by calling *opener* with (*file*, *flags*). *opener* must return an open file descriptor (passing `os.open` as *opener* results in functionality similar to passing `None`).

The newly created file is [non-inheritable](#).

The following example uses the `dir_fd` parameter of the `os.open()` function to open a file relative to a given directory:

```
>>> import os
>>> dir_fd = os.open('somedir', os.O_RDONLY)
>>> def opener(path, flags):
...     return os.open(path, flags, dir_fd=dir_fd)
...
>>> with open('spamspam.txt', 'w', opener=opener) as f:
...     print('This will be written to somedir/spamspam.txt', file=f)
...
>>> os.close(dir_fd) # don't leak a file descriptor
```

The type of [file object](#) returned by the `open()` function depends on the mode. When `open()` is used to open a file in a text mode (`'w'` , `'r'` , `'wt'` , `'rt'` , etc.), it returns a subclass of [io.TextIOBase](#) (specifically [io.TextIOWrapper](#)). When used to open a file in a binary mode with buffering, the returned class is a subclass of [io.BufferedIOBase](#) . The exact class varies: in read binary mode, it returns an [io.BufferedReader](#) ; in write binary and append binary modes, it returns an [io.BufferedWriter](#) , and in read/write mode, it returns an [io.BufferedRandom](#) . When buffering is disabled, the raw stream, a subclass of [io.RawIOBase](#) , [io.FileIO](#) , is returned.

See also the file handling modules, such as [fileinput](#) , [io](#) (where `open()` is declared), [os](#) , [os.path](#) , [tempfile](#) , and [shutil](#) .

Raises an [auditing event](#) `open` with arguments `path` , `mode` , `flags` .

The `mode` and `flags` arguments may have been modified or inferred from the original call.

Changed in version 3.3:

- The `opener` parameter was added.
- The `'x'` mode was added.
- [IOError](#) used to be raised, it is now an alias of [OSError](#) .
- [FileExistsError](#) is now raised if the file opened in exclusive creation mode (`'x'`) already exists.

Changed in version 3.4:

- The file is now non-inheritable.

Changed in version 3.5:

- If the system call is interrupted and the signal handler does not raise an exception, the function now retries the system call instead of raising an [InterruptedError](#) exception (see [PEP 475](#) for the rationale).
- The `'namereplace'` error handler was added.

Changed in version 3.6:

- Support added to accept objects implementing [os.PathLike](#) .
- On Windows, opening a console buffer may return a subclass of [io.RawIOBase](#) other than [io.FileIO](#) .

Changed in version 3.11: The `'U'` mode has been removed.

`ord(character, /)`

Return the ordinal value of a character.

If the argument is a one-character string, return the Unicode code point of that character. For example, `ord('a')` returns the integer `97` and `ord('€')` (Euro sign) returns `8364`. This is the inverse of `chr()`.

If the argument is a `bytes` or `bytearray` object of length 1, return its single byte value. For example, `ord(b'a')` returns the integer `97`.

`pow(base, exp, mod=None)`

Return *base* to the power *exp*; if *mod* is present, return *base* to the power *exp*, modulo *mod* (computed more efficiently than `pow(base, exp) % mod`). The two-argument form `pow(base, exp)` is equivalent to using the power operator: `base**exp`.

When arguments are builtin numeric types with mixed operand types, the coercion rules for binary arithmetic operators apply. For `int` operands, the result has the same type as the operands (after coercion) unless the second argument is negative; in that case, all arguments are converted to float and a float result is delivered. For example, `pow(10, 2)` returns `100`, but `pow(10, -2)` returns `0.01`. For a negative base of type `int` or `float` and a non-integral exponent, a complex result is delivered. For example, `pow(-9, 0.5)` returns a value close to `3j`. Whereas, for a negative base of type `int` or `float` with an integral exponent, a float result is delivered. For example, `pow(-9, 2.0)` returns `81.0`.

For `int` operands *base* and *exp*, if *mod* is present, *mod* must also be of integer type and *mod* must be nonzero. If *mod* is present and *exp* is negative, *base* must be relatively prime to *mod*. In that case, `pow(inv_base, -exp, mod)` is returned, where *inv_base* is an inverse to *base* modulo *mod*.

Here's an example of computing an inverse for `38` modulo `97`:

```
>>> pow(38, -1, mod=97)
23
>>> 23 * 38 % 97 == 1
True
```

Changed in version 3.8: For `int` operands, the three-argument form of `pow` now allows the second argument to be negative, permitting computation of modular inverses.

Changed in version 3.8: Allow keyword arguments. Formerly, only positional arguments were supported.

```
print(*objects, sep=' ', end='\n', file=None, flush=False)
```

Print *objects* to the text stream *file*, separated by *sep* and followed by *end*. *sep*, *end*, *file*, and *flush*, if present, must be given as keyword arguments.

All non-keyword arguments are converted to strings like `str()` does and written to the stream, separated by *sep* and followed by *end*. Both *sep* and *end* must be strings; they can also be `None`, which means to use the default values. If no *objects* are given, `print()` will just write *end*.

The *file* argument must be an object with a `write(string)` method; if it is not present or `None`, `sys.stdout` will be used. Since printed arguments are converted to text strings, `print()` cannot be used with binary mode file objects. For these, use `file.write(...)` instead.

Output buffering is usually determined by *file*. However, if *flush* is true, the stream is forcibly flushed.

Changed in version 3.3: Added the *flush* keyword argument.

`class property(fget=None, fset=None, fdel=None, doc=None)`

Return a property attribute.

fget is a function for getting an attribute value. *fset* is a function for setting an attribute value. *fdel* is a function for deleting an attribute value. And *doc* creates a docstring for the attribute.

A typical use is to define a managed attribute `x`:

```
class C:
    def __init__(self):
        self._x = None

    def getx(self):
        return self._x

    def setx(self, value):
        self._x = value

    def delx(self):
        del self._x

x = property(getx, setx, delx, "I'm the 'x' property.")
```

If *c* is an instance of *C*, `c.x` will invoke the getter, `c.x = value` will invoke the setter, and `del c.x` the deleter.

If given, *doc* will be the docstring of the property attribute. Otherwise, the property will copy *fget*'s docstring (if it exists). This makes it possible to create read-only properties easily using `property()` as a [decorator](#):

```
class Parrot:
    def __init__(self):
        self._voltage = 100000

    @property
    def voltage(self):
        """Get the current voltage."""
        return self._voltage
```

The `@property` decorator turns the `voltage()` method into a “getter” for a read-only attribute with the same name, and it sets the docstring for *voltage* to “Get the current voltage.”

@getter

@setter

A property object has `getter`, `setter`, and `deleter` methods usable as decorators that create a copy of the property with the corresponding accessor function set to the decorated function. This is best explained with an example:

```
class C:
    def __init__(self):
        self._x = None

    @property
    def x(self):
        """I'm the 'x' property."""
        return self._x

    @x.setter
    def x(self, value):
        self._x = value

    @x.deleter
    def x(self):
        del self._x
```

This code is exactly equivalent to the first example. Be sure to give the additional functions the same name as the original property (`x` in this case.)

The returned property object also has the attributes `fget`, `fset`, and `fdel` corresponding to the constructor arguments.

Changed in version 3.5: The docstrings of property objects are now writeable.

`__name__`

Attribute holding the name of the property. The name of the property can be changed at runtime.

Added in version 3.13.

`class range(stop, /)`

`class range(start, stop, step=1, /)`

Rather than being a function, `range` is actually an immutable sequence type, as documented in [Ranges](#) and [Sequence Types — list, tuple, range](#).

`repr(object, /...)`

Return a string containing a printable representation of an object. For many types, this function makes an attempt to return a string that would yield an object with the same value when passed to `eval()`; otherwise, the representation is a string enclosed in angle brackets that contains the name of the type of the object together with additional information often including the name and address of the object. A class can control what this function returns for its instances by defining a `__repr__()` method. If `sys.displayhook()` is not accessible, this function will raise `RuntimeError`.

This class has a custom representation that can be evaluated:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"Person('{self.name}', {self.age})"
```

`reversed(object, /...)`

Return a reverse [iterator](#). The argument must be an object which has a `__reversed__()` method or supports the sequence protocol (the `__len__()` method and the `__getitem__()` method with integer arguments starting at 0).

`round(number, ndigits=None)`

Return *number* rounded to *ndigits* precision after the decimal point. If *ndigits* is omitted or is `None`, it returns the nearest integer to its input.

For the built-in types supporting `round()`, values are rounded to the closest multiple of 10 to the power minus *ndigits*; if two multiples are equally close, rounding is done toward the even choice (so, for example, both `round(0.5)` and `round(-0.5)` are 0, and `round(1.5)` is 2). Any integer value is valid for *ndigits* (positive, zero, or negative). The return value is an integer if *ndigits* is omitted or `None`. Otherwise, the return value has the same type as *number*.

For a general Python object `number`, `round` delegates to `number.__round__`.

Note: The behavior of `round()` for floats can be surprising: for example, `round(2.675, 2)` gives `2.67` instead of the expected `2.68`. This is not a bug: it's a result of the fact that most decimal fractions can't be represented exactly as a float. See [Floating-Point Arithmetic: Issues and Limitations](#) for more information.

`class set(iterable=(), /)`

Return a new `set` object, optionally with elements taken from *iterable*. `set` is a built-in class. See `set` and [Set Types — set, frozenset](#) for documentation about this class.

For other containers see the built-in `frozenset`, `list`, `tuple`, and `dict` classes, as well as the `collections` module.

`setattr(object, name, value, /)`

This is the counterpart of `getattr()`. The arguments are an object, a string, and an arbitrary value. The string may name an existing attribute or a new attribute. The function assigns the value to the attribute, provided the object allows it. For example, `setattr(x, 'foobar', 123)` is equivalent to `x.foobar = 123`.

name need not be a Python identifier as defined in [Names \(identifiers and keywords\)](#) unless the object chooses to enforce that, for example in a custom `__getattr__()` or via `__slots__`. An attribute whose name is not an identifier will not be accessible using the dot notation, but is accessible through `getattr()` etc..

Note: Since [private name mangling](#) happens at compilation time, one must manually mangle a private attribute's (attributes with two leading underscores) name in order to set it with `setattr()`.

```
class slice(stop, /)
```

```
class slice(start, stop, step=None, /)
```

Return a [slice](#) object representing the set of indices specified by `range(start, stop, step)`. The *start* and *step* arguments default to `None`.

Slice objects are also generated when [slicing syntax](#) is used. For example: `a[start:stop:step]` or `a[start:stop, i]`.

See [itertools.islice\(\)](#) for an alternate version that returns an [iterator](#).

start

stop

step

These read-only attributes are set to the argument values (or their default). They have no other explicit functionality; however, they are used by NumPy and other third-party packages.

Changed in version 3.12: Slice objects are now [hashable](#) (provided [start](#), [stop](#), and [step](#) are hashable).

`sorted(iterable, /, *, key=None, reverse=False)`

Return a new sorted list from the items in *iterable*.

Has two optional arguments which must be specified as keyword arguments.

key specifies a function of one argument that is used to extract a comparison key from each element in *iterable* (for example, `key=str.lower`). The default value is `None` (compare the elements directly).

reverse is a boolean value. If set to `True`, then the list elements are sorted as if each comparison were reversed.

Use `functools.cmp_to_key()` to convert an old-style *cmp* function to a *key* function.

The built-in `sorted()` function is guaranteed to be stable. A sort is stable if it guarantees not to change the relative order of elements that compare equal — this is helpful for sorting in multiple passes (for example, sort by department, then by salary grade).

The sort algorithm uses only `<` comparisons between items. While defining an `__lt__()` method will suffice for sorting, [PEP 8](#) recommends that all six [rich comparisons](#) be implemented. This will help avoid bugs when using the same data with other ordering tools such as `max()` that rely on a different underlying method. Implementing all six comparisons also helps avoid confusion for mixed type comparisons which can call the reflected `__gt__()` method.

For sorting examples and a brief sorting tutorial, see [Sorting Techniques](#).

@staticmethod

Transform a method into a static method.

A static method does not receive an implicit first argument. To declare a static method, use this idiom:

```
class C:
    @staticmethod
    def f(arg1, arg2, argN): ...
```

The `@staticmethod` form is a function [decorator](#) – see [Function definitions](#) for details.

A static method can be called either on the class (such as `C.f()`) or on an instance (such as `C().f()`). Moreover, the static method [descriptor](#) is also callable, so it can be used in the class definition (such as `f()`).

Static methods in Python are similar to those found in Java or C++. Also, see [classmethod\(\)](#) for a variant that is useful for creating alternate class constructors.

Like all decorators, it is also possible to call `staticmethod` as a regular function and do something with its result. This is needed in some cases where you need a reference to a function from a class body and you want to avoid the automatic transformation to instance method. For these cases, use this idiom:

```
def regular_function():
    ...

class C:
    method = staticmethod(regular_function)
```

For more information on static methods, see [The standard type hierarchy](#).

Changed in version 3.10: Static methods now inherit the method attributes (`__module__`, `__name__`, `__qualname__`, `__doc__` and `__annotations__`), have a new `__wrapped__` attribute, and are now callable as regular functions.

```
class str(*, encoding='utf-8', errors='strict')
```

```
class str(object)
```

```
class str(object, encoding, errors='strict')
```

```
class str(object, *, errors)
```

Return a `str` version of *object*. See `str()` for details.

`str` is the built-in string [class](#). For general information about strings, see [Text Sequence Type — str](#).

```
sum(iterable, /, start=0)
```

Sums *start* and the items of an *iterable* from left to right and returns the total. The *iterable*'s items are normally numbers, and the start value is not allowed to be a string.

For some use cases, there are good alternatives to `sum()`. The preferred, fast way to concatenate a sequence of strings is by calling `''.join(sequence)`. To add floating-point values with extended precision, see `math.fsum()`. To concatenate a series of iterables, consider using `itertools.chain()`.

Changed in version 3.8: The *start* parameter can be specified as a keyword argument.

Changed in version 3.12: Summation of floats switched to an algorithm that gives higher accuracy and better commutativity on most builds.

Changed in version 3.14: Added specialization for summation of complexes, using same algorithm as for summation of floats.

`class super`

`class super(type, object_or_type=None, /)`

Return a proxy object that delegates method calls to a parent or sibling class of `type`. This is useful for accessing inherited methods that have been overridden in a class.

The `object_or_type` determines the [method resolution order](#) to be searched. The search starts from the class right after the `type`.

For example, if `__mro__` of `object_or_type` is `D -> B -> C -> A -> object` and the value of `type` is `B`, then `super()` searches `C -> A -> object`.

The `__mro__` attribute of the class corresponding to `object_or_type` lists the method resolution search order used by both `getattr()` and `super()`. The attribute is dynamic and can change whenever the inheritance hierarchy is updated.

If the second argument is omitted, the super object returned is unbound. If the second argument is an object, `isinstance(obj, type)` must be true. If the second argument is a type, `issubclass(type2, type)` must be true (this is useful for classmethods).

When called directly within an ordinary method of a class, both arguments may be omitted (“zero-argument `super()`”). In this case, `type` will be the enclosing class, and `obj` will be the first argument of the immediately enclosing function (typically `self`). (This means that zero-argument `super()` will not work as expected within nested functions, including generator expressions, which implicitly create nested functions.)

There are two typical use cases for `super`. In a class hierarchy with single inheritance, `super` can be used to refer to parent classes without naming them explicitly, thus making the code more maintainable. This use closely parallels the use of `super` in other programming languages.

The second use case is to support cooperative multiple inheritance in a dynamic execution environment. This use case is unique to Python and is not found in statically compiled languages or languages that only support single inheritance. This makes it possible to implement “diamond diagrams” where multiple base classes implement the same method. Good design dictates that such implementations have the same calling signature in every case (because the order of calls is determined at runtime, because that order adapts to changes in the class hierarchy, and because that order can include sibling classes that are unknown prior to runtime).

For both use cases, a typical superclass call looks like this:

```
class C(B):
    def method(self, arg):
        super().method(arg)    # This does the same thing as:
                               # super(C, self).method(arg)
```

In addition to method lookups, `super()` also works for attribute lookups. One possible use case for this is calling [descriptors](#) in a parent or sibling class.

Note that `super()` is implemented as part of the binding process for explicit dotted attribute lookups such as `super().__getitem__(name)`. It does so by implementing its own `__getattr__()` method for searching classes in a predictable order that supports cooperative multiple inheritance. Accordingly, `super()` is undefined for implicit lookups using statements or operators such as `super()[name]`.

Also note that, aside from the zero argument form, `super()` is not limited to use inside methods. The two argument form specifies the arguments exactly and makes the appropriate references. The zero argument form only works inside a class definition, as the compiler fills in the necessary details to correctly retrieve the class being defined, as well as accessing the current instance for ordinary methods.

For practical suggestions on how to design cooperative classes using `super()`, see [guide to using super\(\)](#).

Changed in version 3.14: `super` objects are now `pickleable` and `copyable`.

```
class tuple(iterable=(), ...)
```

Rather than being a function, `tuple` is actually an immutable sequence type, as documented in [Tuples](#) and [Sequence Types — list, tuple, range](#).

`class type(object, __)`

`class type(name, bases, dict, __, **kwargs)`

With one argument, return the type of an *object*. The return value is a type object and generally the same object as returned by `object.__class__`.

The `isinstance()` built-in function is recommended for testing the type of an object, because it takes subclasses into account.

With three arguments, return a new type object. This is essentially a dynamic form of the `class` statement. The *name* string is the class name and becomes the `__name__` attribute. The *bases* tuple contains the base classes and becomes the `__bases__` attribute; if empty, `object`, the ultimate base of all classes, is added. The *dict* dictionary contains attribute and method definitions for the class body; it may be copied or wrapped before becoming the `__dict__` attribute. The following two statements create identical `type` objects:

```
>>> class X:
...     a = 1
...
>>> X = type('X', (), dict(a=1))
```

See also:

- [Documentation on attributes and methods on classes](#).
- [Type Objects](#)

Keyword arguments provided to the three argument form are passed to the appropriate metaclass machinery (usually `__init_subclass__()`) in the same way that keywords in a class definition (besides *metaclass*) would.

See also [Customizing class creation](#).

Changed in version 3.6: Subclasses of `type` which don't override `type.__new__` may no longer use the one-argument form to get the type of an object.

`vars()`

`vars(object, /)`

Return the `__dict__` attribute for a module, class, instance, or any other object with a `__dict__` attribute.

Objects such as modules and instances have an updateable `__dict__` attribute; however, other objects may have write restrictions on their `__dict__` attributes (for example, classes use a `types.MappingProxyType` to prevent direct dictionary updates).

Without an argument, `vars()` acts like `locals()`.

A `TypeError` exception is raised if an object is specified but it doesn't have a `__dict__` attribute (for example, if its class defines the `__slots__` attribute).

Changed in version 3.13: The result of calling this function without an argument has been updated as described for the `locals()` builtin.

zip(*iterables, strict=False)

Iterate over several iterables in parallel, producing tuples with an item from each one.

Example:

```
>>> for item in zip([1, 2, 3], ['sugar', 'spice', 'everything nice']):
...     print(item)
...
(1, 'sugar')
(2, 'spice')
(3, 'everything nice')
```

More formally: `zip()` returns an iterator of tuples, where the i -th tuple contains the i -th element from each of the argument iterables.

Another way to think of `zip()` is that it turns rows into columns, and columns into rows. This is similar to [transposing a matrix](#).

`zip()` is lazy: The elements won't be processed until the iterable is iterated on, e.g. by a `for` loop or by wrapping in a `list`.

One thing to consider is that the iterables passed to `zip()` could have different lengths; sometimes by design, and sometimes because of a bug in the code that prepared these iterables. Python offers three different approaches to dealing with this issue:

- By default, `zip()` stops when the shortest iterable is exhausted. It will ignore the remaining items in the longer iterables, cutting off the result to the length of the shortest iterable:

```
>>> list(zip(range(3), ['fee', 'fi', 'fo', 'fum']))
[(0, 'fee'), (1, 'fi'), (2, 'fo')]
```

- `zip()` is often used in cases where the iterables are assumed to be of equal length. In such cases, it's recommended to use the `strict=True` option. Its output is the same as regular `zip()`:

```
>>> list(zip(('a', 'b', 'c'), (1, 2, 3), strict=True))
[('a', 1), ('b', 2), ('c', 3)]
```

Unlike the default behavior, it raises a `ValueError` if one iterable is exhausted before the others:

```
>>> for item in zip(range(3), ['fee', 'fi', 'fo', 'fum'], strict=True):
...     print(item)
...
(0, 'fee')
(1, 'fi')
(2, 'fo')
Traceback (most recent call last):
...
ValueError: zip() argument 2 is longer than argument 1
```

Without the `strict=True` argument, any bug that results in iterables of different lengths will be silenced, possibly manifesting as a hard-to-find bug in another part of the program.

- Shorter iterables can be padded with a constant value to make all the iterables have the same length. This is done by `itertools.zip_longest()` .

Edge cases: With a single iterable argument, `zip()` returns an iterator of 1-tuples. With no arguments, it returns an empty iterator.

Tips and tricks:

- The left-to-right evaluation order of the iterables is guaranteed. This makes possible an idiom for clustering a data series into n-length groups using `zip(*[iter(s)]*n, strict=True)` . This repeats the *same* iterator `n` times so that each output tuple has the result of `n` calls to the iterator. This has the effect of dividing the input into n-length chunks.
- `zip()` in conjunction with the `*` operator can be used to unzip a list:

```
>>> x = [1, 2, 3]
>>> y = [4, 5, 6]
>>> list(zip(x, y))
[(1, 4), (2, 5), (3, 6)]
>>> x2, y2 = zip(*zip(x, y))
>>> x == list(x2) and y == list(y2)
True
```

Changed in version 3.10: Added the `strict` argument.

`__import__(name, globals=None, locals=None, fromlist=(), level=0)`

Note: This is an advanced function that is not needed in everyday Python programming, unlike `importlib.import_module()`.

This function is invoked by the `import` statement. It can be replaced (by importing the `builtins` module and assigning to `builtins.__import__`) in order to change semantics of the `import` statement, but doing so is **strongly** discouraged as it is usually simpler to use import hooks (see [PEP 302](#)) to attain the same goals and does not cause issues with code which assumes the default import implementation is in use. Direct use of `__import__()` is also discouraged in favor of `importlib.import_module()`.

The function imports the module *name*, potentially using the given *globals* and *locals* to determine how to interpret the name in a package context. The *fromlist* gives the names of objects or submodules that should be imported from the module given by *name*. The standard implementation does not use its *locals* argument at all and uses its *globals* only to determine the package context of the `import` statement.

level specifies whether to use absolute or relative imports. `0` (the default) means only perform absolute imports. Positive values for *level* indicate the number of parent directories to search relative to the directory of the module calling `__import__()` (see [PEP 328](#) for the details).

When the *name* variable is of the form `package.module`, normally, the top-level package (the name up till the first dot) is returned, *not* the module named by *name*. However, when a non-empty *fromlist* argument is given, the module named by *name* is returned.

For example, the statement `import spam` results in bytecode resembling the following code:

```
spam = __import__('spam', globals(), locals(), [], 0)
```

The statement `import spam.ham` results in this call:

```
spam = __import__('spam.ham', globals(), locals(), [], 0)
```

Note how `__import__()` returns the toplevel module here because this is the object that is bound to a name by the `import` statement.

On the other hand, the statement `from spam.ham import eggs, sausage as saus` results in

```
_temp = __import__('spam.ham', globals(), locals(), ['eggs', 'sausage'], 0)
eggs = _temp.eggs
saus = _temp.sausage
```

Here, the `spam.ham` module is returned from `__import__()`. From this object, the names to import are retrieved and assigned to their respective names.

If you simply want to import a module (potentially within a package) by name, use `importlib.import_module()`.

Changed in version 3.3: Negative values for *level* are no longer supported (which also changes the default value to 0).

Changed in version 3.9: When the command line options `-E` or `-I` are being used, the environment variable `PYTHONCASEOK` is now ignored.

Footnotes

- [1] Note that the parser only accepts the Unix-style end of line convention. If you are reading the code from a file, make sure to use newline conversion mode to convert Windows or Mac-style newlines.